cartcell MHC-II predictor — 전체 히스토리와 현재 상황

최종 갱신: **2026-11-11**

이 문서는 프로젝트의 **단일 기준 문서**입니다. 서버 유실 이후의 로컬 재구성, 세운 가설, 그 가설을 검증한 실험, 그리고 **가설이 기각된 경위**까지를 숫자와 근거 파일 경로로 남깁니다.

같은 폴더의 다른 두 문서는 성격이 다릅니다.

- `cartcell_rebuild_briefing.md` — 서버 유실 직후, 대화 기록에서 복원한 **스펙 추정치**. 검증 전 자료.
- `cartcell_server_state.md` — 유실된 서버 환경의 상태 기록.

아래 숫자는 모두 이 저장소의 `reports_*` 산출물에서 직접 읽은 값입니다. **틀렸다고 밝혀진 서술은 지우지 않고 정정 표시와 함께 남깁니다** — 어느 숫자가 어떤 이유로 바뀌었는지가 기록의 핵심입니다.

---

0. 한 줄 요약

원래 가설은 기각됐습니다. 그러나 그 자리에 더 구체적이고 검증 가능한 결론이 섰습니다: CapsNet은 binding core를 식별해서 잘하는 게 아니고, 회귀에서는 이기지만 **그 이득은 dynamic routing에서 오지 않습니다**(+0.002, p=0.373). 그리고 왜 오지 않는지 메커니즘까지 측정했습니다.

[정정] 2026-11-11 — 회귀 격차의 크기는 현재 신뢰할 수 없습니다. 수렴 프로브에서 50 epoch 예산이 **Phase2를 훨씬 더 심하게 굶긴 것**이 드러났습니다. 공정한 예산에서는 격차가 **+0.128 → +0.037로 줄고**, 복합 지표로는 Phase2가 역전합니다(§5.6). **§5의 표와 §7의 서술은 5-fold 재학습 뒤에 다시 써야 합니다.** 기각된 가설(core 식별)과 routing null 결과(+0.002, p=0.373)는 **영향 없습니다** — 둘 다 예산과 무관한 비교이고, routing null은 `|w|`가 6–10배 커진 뒤에도 유지됩니다.

---

1. 배경

1.1 무엇을 예측하는가

MHC-II 분자와 peptide의 결합 친화도. 13–25 aa peptide가 주어지면 그 안의 **9-mer binding core**가 MHC-II groove에 들어가고, 양 옆은 flank로 남습니다. 어느 9-mer가 core인지는 **라벨이 없습니다** — 이게 이 문제의 본질적 어려움입니다.

라벨 변환:

```
affinity = 1 - log(IC50) / log(50000)
```

1.2 데이터

- `custom_dataset/Anthem_dataset/` — train 25,628 / test 6,407 쌍
- 68 MHC-II allele (register-shift 실험 기준), HLA-DR/DP/DQ
- strong binder(≤500 nM) 비율 약 5% → `pos_weight ~ 18.5`로 weighted BCE
- 500 nM 임계값은 affinity 척도로 **0.4256**, 50 nM은 **0.6384**

1.3 모델 두 개

	구조
Phase2 (CNN)	CNN + CoreRegister. T=17개 후보 위치에 softmax(pos_prob) → 가중합 core_vec. core_conditioning으로 allele 정보를 core 선택에 주입

	구조
Phase3 (CapsNet)	multi-branch primary capsule 56개(full 32 + core 16 + MHC 8) + dynamic routing → 3 output capsule. Phase2에서 core/MHC 가중치 35개 텐서를 warm-start

pseudo-sequence는 실서열입니다. `custom_dataset/Anthem_dataset/mhc_ii_pseudo.csv` 68행 전부 `source=real` (34-mer). 과거의 placeholder 더미 단계는 지나갔습니다.

`register_shift_summary.json`의 `pseudo_sequence_warning`이 한동안 `"0/68 alleles use PLACEHOLDER ... carries no real signal"`로 출력되어 **placeholder로 돌린 run처럼 읽혔습니다.** 경고 문장이 카운트와 무관하게 무조건 붙던 버그이고, 지금은 카운트가 0이면 문장도 달라지게 고쳤습니다. **숫자 0/68이 맞는 값이고, 이 run들은 실서열 기반입니다.**

---

2. 검증하려던 가설

CapsNet + dynamic routing은 binding core(register)를 일관되게 식별하며, 이는 CNN보다 정확하고, 그 능력은 dynamic routing에서 나온다.

지도교수 피드백으로 이 문장이 **세 개의 독립적 주장**으로 쪼개진다는 걸 확인했고, 세 개를 각각 측정했습니다.

1. core를 일관되게 식별한다
2. CNN보다 정확하게 추적한다
3. (숨은 전제) 그게 dynamic routing 덕분이다

---

3. 실험 설계 — register-shift

핵심 함정: **불변성만 재면 아무 의미가 없습니다.** 입력을 무시하는 상수 모델이 불변성 100%를 받습니다. 그래서 **민감도 대조군을 함께** 넣었습니다.

`codes/custom_codes/register_shift.py`, 조건당 300 케이스:

조건	무엇을 바꾸나	기대
shift_flank	**core 위치만 옮김 (flank 길이 재분배)**	**예측이 안 흔들려야**
shift_pad	**padding으로 위치만 옮김**	**예측이 안 흔들려야**
flank_resample	**위치 고정, flank 서열만 교체**	**shift와 비교할 기준선**
core_mutate	**core 내부 잔기 변이**	**예측이 흔들려야**
scramble_all	**전체 서열 셔플 (조성 보존)**	**예측이 흔들려야**

누출 감사도 함께 돌렸습니다: `exact_pair_in_train = 0` / 16,468 variant. `core9_in_train_same_allele`이 1,783로 높지만 이건 **설계상 의도된 것**입니다 — anchor가 실제 peptide를 재사용하므로. 이 실험은 **예측 안정성**을 재는 것이고 미지 core에 대한 일반화를 재는 게 아닙니다.

---

4. 결과 (1) — 가설 세 조항 전부 기각

4.1 core를 일관되게 식별하는가 → 아니다

`shift_flank`에서 **같은 9-mer를 짓는 비율(same_core_rate)**:

모델	same_core_rate	fixed_index_rate
Phase2 (CNN)	**0.412**	**0.050**
Phase3 (routing 3)	**0.403**	**0.053**

모델	same_core_rate	fixed_index_rate
Phase3 no-routing	**0.420**	**0.045**

랜덤 9-mer 일치 기대값(null)이 **0.416**입니다. 세 모델 전부 null과 구분되지 않습니다. `fixed_index_rate`가 0.05로 낮으니 "그냥 고정 위치를 집는다"는 경쟁 설명도 아닙니다 — **위치를 옮기면 다른 9-mer를 집습니다.**

`shift_pad`에서는 0.51–0.54로 조금 높지만(drift 1.80–2.06), 이건 padding이 문맥을 덜 바꾸기 때문이고 여전히 "일관되게 식별한다"고 부를 수준이 아닙니다.

중요: core 추적이 불안정한 것과 **예측 자체가 안정한 것**은 별개입니다. 예측은 안정합니다(`shift_flank` / `flank_resample` 변동 비 0.88–0.95). 모델은 register를 명시적으로 식별하지 않고도 안정적으로 예측합니다.

4.2 CNN보다 정확한가 → 아니다

0.412(Phase2) vs 0.403(Phase3). 차이 없습니다.

기록해 둘 사실: 이 비교표가 한때 **뒤집혀 있었습니다.** 0.412가 Phase3 숫자라고 적힌 적이 있는데, `register_shift_summary.json`의 `args.ckpt_dir`을 확인한 결과 `reports_phase2_final/checkpoints` — **0.412는 Phase2**가 맞습니다. 또 "이 숫자들은 `core_conditioning` 적용 전"이라는 지적이 있었으나, 체크포인트의 `cond_gate` 값(0.351 / 0.251 / 0.228 / 0.352 / -0.216)이 0이 아니므로 **적용 후**의 숫자입니다.

4.3 dynamic routing 덕분인가 → 아니다

`config/phase3_norouting.json` — `routing_iters: 1`만 다르고 나머지는 전부 동일(diff로 확인). `n_iters=1`이면 b가 0에 머물러 `c = softmax(0) = 1/3` **정확히 균일**, 즉 `u_hat`의 단순 평균이 됩니다. **W와 squash와 capsule 구조는 그대로 학습되므로, 이건 "routing 반복"만 제거하는 올바른 ablation**입니다.

→ 모든 지표에서 full routing과 구분되지 않음. 자세한 수치는 §5.

4.4 추가로 드러난 것 — Phase3는 민감도 대조군을 통과하지 못한다

`scramble_all / shift_flank` 변동 비 (**1보다 커야** 서열에 민감한 것):

모델	비	해석
Phase2 (CNN)	**1.48**	**통과 (p = 5e-31)**
Phase3 (routing 3)	**0.81**	**실패, 유의하지 않음**
Phase3 no-routing	**0.82**	**실패**

`scramble_all`은 **아미노산 조성을 보존**합니다. Phase2는 셔플하면 흔들리는데 Phase3는 오히려 덜 흔들립니다 → **Phase3는 조성/길이 같은 더 거친 특징에 의존**한다는 증거. Phase3의 기준선 wobble도 Phase2의 2배입니다.

`core_mutate / shift_flank` 비는 Phase2 2.32, Phase3 1.67, no-routing 1.60 — core 변이에는 세 모델 다 반응하지만 Phase2가 가장 강합니다.

---

5. 결과 (2) — 공정 비교 (round 2, 최종)

5.1 왜 다시 돌렸나: 두 개의 교란 요인

(a) 체크포인트 선택 기준이 불안정했습니다. weighted BCE val_loss는 ranking 지표가 좋아지는 동안에도 올라갑니다(이미 알려진 `phase2-early-stopping-bug`: val_loss로 고르면 epoch 0을 고름). 그래서 AUPRC로 바꿨었는데, AUPRC도 흔들렸습니다. AUROC 단독은 **fold 2, 3에서 epoch 0**을 골라 더 나빴습니다. capsule 모델에서는 AUROC가 epoch 0–1에 피크를 찍고 내려가는 반면 Pearson r은 epoch 38에도 여전히 오르고 있어서, **어느 한 지표로도 안 됩니다.**

→ `train_phase2.py`에 공유 헬퍼 `selection_score()`를 넣고 **`mean(AUROC, Pearson r)` 복합 기준**을 도입.

`train_phase3.py`와 `lomo.py`가 같이 씁니다.

(b) λ_reg가 모델 간에 달랐습니다 — 회귀 손실 가중치가 다른데 회귀 성능을 비교하고 있었습니다.

→ 셋 다 **λ_reg = 0.5, epochs = 50, 복합 기준**으로 통일해 재학습.

```
Phase2 (CNN)        lam=0.5 ep=50 sel=['auroc','pearson_r'] routing=-
Phase3 (routing 3)  lam=0.5 ep=50 sel=['auroc','pearson_r'] routing=3
no-routing          lam=0.5 ep=50 sel=['auroc','pearson_r'] routing=1
```

5.2 성능

모델	split	AUROC	AUPRC	Pearson r	MSE
Phase2 (CNN)	oof	0.861	0.379	0.510	0.0531
Phase2 (CNN)	test	0.895	0.426	0.552	0.0506
Phase3 (routing 3)	oof	0.824	0.282	0.634	0.0429
Phase3 (routing 3)	test	0.882	0.373	0.680	0.0382
no-routing (1)	oof	0.826	0.300	0.638	0.0428
no-routing (1)	test	0.883	0.347	0.682	0.0380

5.3 부트스트랩 검정 (2,000 resample, 동일 test 행)

행 정렬은 `affinity` 컬럼 `np.allclose`로 검증했습니다.

비교	Pearson r 차이	AUPRC 차이
Phase3 - Phase2	+0.1284 [+0.1165, +0.1400] p<0.001	-0.0516 [-0.0820, -0.0219] p=0.001
no-routing - Phase3	+0.0021 [-0.0024, +0.0069] p=0.373	-0.0247 [-0.0512, +0.0008] p=0.057

절대값:

	Pearson r	95% CI	AUPRC	95% CI
Phase2	0.552	[0.533, 0.569]	0.427	[0.369, 0.481]
Phase3	0.680	[0.665, 0.694]	0.375	[0.320, 0.430]
no-routing	0.682	[0.667, 0.697]	0.351	[0.301, 0.404]

읽는 법:

- **회귀는 CapsNet, 분류는 CNN.** 둘 다 유의하고, 트레이드오프가 실재합니다.
- **routing 기여는 0.** r 차이의 CI가 0을 포함하고 p=0.373. AUPRC만 -0.025로 경계선(p=0.057)인데 이 지표는 노이즈가 크고 방향도 routing에 유리해서 주장 근거로 쓰기 어렵습니다. **독립적인 두 번의 확인**입니다 — round 1(λ_reg 0.5 / epoch 40 / AUPRC 선택)과 round 2(현 조건).

5.4 기록해 둘 틀린 예측

재학습 전에 이렇게 예측했습니다:

"Phase3의 회귀 우위가 상당 부분 λ_reg였을 가능성이 크고, 크게 축소될 것이다."

틀렸습니다.

Pearson r	λ_reg 불일치 (0.1 / 0.5)	λ_reg 통일 (0.5)
Phase2	0.483	0.552 (+0.069)
Phase3	0.608	0.680 (+0.072)
격차	0.125	0.128

λ_reg를 올리자 **두 모델이 거의 같은 폭으로 같이 올라갔습니다.** Phase2만 오를 거라고 가정한 게 잘못이었습니다. 결과적으로 **Phase3의 회귀 우위는 아키텍처 효과로 확정**됐고, 이제는 통제된 조건에서 얻은, CI가 0을 한참 벗어나는

숫자입니다.

5.5 남은 흠 — epoch 예산

`best_epoch`이 여전히 50 예산에 붙어 있습니다:

모델	best_epoch / fold
Phase2	42, 37, 47, 35, 44
Phase3	48, 41, 38, 34, 47
no-routing	49, 49, 39, 34, 47

50 epoch도 부족합니다. 처음에는 **"세 모델이 똑같이 부족하므로 비교의 공정성은 유지된다"**고 적었는데, **2026-11-11 수렴 프로브로 반증됐습니다 → §5.6.**

5.6 수렴 프로브 (2026-11-11) — 예산이 §5.2의 격차를 만들었습니다

fold 0 하나를 세 모델 모두 150 epoch까지 돌리며 매 epoch을 기록했습니다 (`run_convergence_probe.sh`, 트레이스는 `reports_probe_*/trace/`).

세 모델은 똑같이 부족하지 않았습니다. Phase2가 압도적으로 더 굶었습니다.

fold 0 val	r @49 (구 예산)	r @best	Δ	best_epoch	평평해지는 epoch
Phase2	0.523	0.658	+0.135	146	아직 상승 중
Phase3	0.655	0.694	+0.039	125	125
no-routing	0.654	0.683	+0.029	124	113

그래서 Phase3 - Phase2 회귀 격차가 72% 줄어듭니다:

	Phase3 - Phase2 (Pearson r)
@49 — 구 예산	+0.132
@best — 공정 예산	+0.037

@49의 +0.132가 §5.3이 test에서 보고한 **+0.128과 거의 일치**합니다. fold 0 val 궤적이 보고된 비교를 제대로 대리한다는 뜻이고, 그래서 +0.037로의 축소를 신뢰할 수 있습니다.

방향이 바뀐 것은 없지만 크기는 바뀝니다. 분류 격차는 오히려 **벌어집니다** (AUROC @best — Phase2 0.860 vs Phase3 0.809). 복합 지표로는 **Phase2가 역전합니다** (0.759 vs 0.751). routing 결론은 **그대로**입니다 — no-routing - Phase3 = -0.011.

지금 상태로는 §5.2/§5.3의 숫자를 논문에 쓸 수 없습니다. Phase2가 150 epoch에서도 단조 상승 중이라(마지막 80 epoch: 0.7125 → 0.7587) 아직 공정한 예산이 어디인지도 모릅니다. Phase2만 400 epoch까지 연장해 평평해지는 지점을 찾는 중이고(1 fold ~ 3.5 h), 그 뒤 세 모델을 그 예산으로 **5-fold 전부 재학습**해야 합니다(250 epoch 기준 합계 ~ 54 h).

---

6. 결과 (3) — routing은 왜 기여가 없는가 (메커니즘)

"routing 제거해도 같다"는 결과에는 전혀 다른 두 설명이 있습니다:

- **(a)** routing은 작동하지만 agreement 신호가 이 과제에 쓸모없다
- **(b)** routing이 `n_iters=1`이 주는 균일 coupling으로 붕괴해서, **애초에 제거할 게 없었다**

학습된 가중치만으로 구분할 수 있습니다. 진단 스크립트 두 개를 새로 썼습니다.

6.1 `diag_coupling.py` — coupling 텐서를 직접 꺼낸다

`model.routing`에 forward hook을 걸어 **(B, 56, 3) 전체 coupling**을 수집하고, 분산을 **입력 의존(within-cell)** vs **고정 패턴(between-cell)** 으로 분해합니다. forward() 출력만으로는 절대 보이지 않는 양입니다.

fold	|c - 1/3|	입력 간 std	cell-mean std	entropy (균일 = 1.0986)
0	0.126	0.056	0.150	0.982
1	0.063	0.027	0.072	1.072
2	0.112	0.043	0.139	1.005
3	0.134	0.049	0.159	0.973
4	0.102	0.039	0.122	1.023

no-routing은 5 fold 전부 **정확히 0.0000 / entropy 1.0986** — 설계대로 균일입니다(검증 완료).

결론: coupling은 균일에서 **벗어납니다**(설명 b는 기각). 그런데 **변동의 87.8–91.4%가 고정된 per-cell 패턴**이고 **입력 의존 성분은 8.6–12.2%뿐**입니다. 고정 재가중은 뒤의 `reg_out`과 norm 기반 분류가 흡수할 수 있습니다 → **그래서 제거해도 비용이 0입니다.**

6.2 `diag_routing_iters.py` — 반복을 늘리면 되는가 → 아니다

학습된 `W`로 routing loop를 손으로 10회까지 재생하며, 매 단계 "c의 변동 중 peptide에 따라 달라지는 비율"을 기록했습니다. 올라가면 예산이 부족했던 것이고, 평평하면 agreement 신호 자체에 정보가 없는 것입니다.

fold 0:

```
 iter  |c-1/3|   input-dep   mean|b|   mean|Δb|
    1  0.00000      (n/a)     0.0000   0.17875
    2  0.06317      19.1%     0.1788   0.18185
    3  0.12550      19.1%     0.3606   0.18329
    5  0.21489      20.1%     0.7271   0.18321
   10  0.30393      21.5%     1.6498   0.19004
```

(iter 1은 b=0이라 분산이 0/0 — 의미 없는 값입니다)

5 fold 전부 iter 3 → iter 10에서 **평평**: 19.1→21.5 / 17.6→17.9 / 12.5→13.7 / 12.1→12.5 / 12.9→13.8 %.

가중치 스케일:

fold	|W|	||u||	||u_hat||
0	0.0362	0.882	0.349
1	0.0195	0.880	0.171
2	0.0322	0.870	0.296
3	0.0374	0.892	0.373
4	0.0336	0.870	0.319

`W`는 **작지만 갇혀 있지는 않습니다.** 초기화는 `0.01·randn`이고 그 평균 절대값은 `0.01·√(2/π) = 0.0080`이므로, 학습 후 0.0195–0.0374는 fold별로 **2.4–4.7배 성장**한 값입니다. 즉 "초기화에서 움직이지 못했다"가 아니라 **"느리게 자라던 중"**입니다. **2026-11-11 프로브로 확정: `W`는 50 epoch에서 자라는 중에 끊긴 것이 맞습니다.** 150 epoch까지 돌리면 `|W|`가 계속 자랍니다 — Phase3 4.5× → **6.1×**, no-routing 6.8× → **9.8×**(성장은 느려짐, Q4/Q1 비 0.17 / 0.23).

그리고 이게 `W` 초기화 스케일 실험을 사실상 불필요하게 만듭니다. 스케일을 키워서 얻고 싶었던 게 "더 큰 `W`에서는 routing이 기여하는가"인데, 예산만 늘려도 `W`가 **init의 6–10배**까지 저절로 커지고 — 그 상태에서도 routing 기여는 여전히 0입니다(no-routing - Phase3 = -0.011). 즉 **routing의 null 결과는 `W`가 6–10배 커진 뒤에도 살아남습니다.** 이건 방어해야 할 약점이 아니라 **주장을 강화하는 측정**입니다.

측정된 결과만 보면: `||u_hat||`(0.17–0.37)이 `||u||`(0.87)보다 훨씬 작고, `Δb`가 반복마다 **거의 같은 값**(0.1788 → 0.1819 → 0.1833 → ...)을 더하기만 합니다.

핵심 진단: 이건 갱신이 아니라 증폭입니다. `b`가 커지면서 `|c-1/3|`은 늘어나지만, 늘어나는 방향이 매 반복 거의 같아서 coupling은 입력과 무관한 고정 패턴을 더 선명하게 만들 뿐입니다. **반복을 늘리는 sweep은 불필요합니다 — 측정으로 반증됐습니다.**

---

7. 논문에 쓸 서술

가설 세 조항은 그대로 기각입니다. 대신 훨씬 구체적인 대체 서술이 섰습니다. **단, 괄호 안의 회귀 숫자는 재학습 뒤에 채워야 합니다**(§5.6):

CapsNet은 register를 식별해서 잘하는 게 아니다. core 추적은 CNN과 같고 (0.403 vs 0.412, null 0.416) 서열 민감도는 오히려 낮다 (scramble 비 0.81× vs 1.48×). 회귀는 더 낫지만 그 폭은 작다 (Pearson r **+0.037**, 공정 예산 1 fold 추정 — 5-fold CI 재산출 필요). 그 이득은 dynamic routing에서 오지 않는다(+0.002, p=0.373) — routing의 coupling은 입력 의존 성분이 9–12%에 불과해 고정 재가중과 구분되지 않으며, 반복을 10회로 늘려도 그 비율이 평평하다. 예산을 3배로 늘려 `W`가 init의 **6–10배**까지 자란 뒤에도 기여는 여전히 0이다. **이득은 capsule 구조 자체에서 오고, 생각보다 작다.**

취소선 그을 문장: 이전 판은 여기에 "회귀는 확실히 낫다 (+0.128, 95% CI [+0.117, +0.140], p<0.001; MSE 25% 감소)"와 "routing을 작동하게 만들면? W 초기화 스케일을 키우는 실험이 답이고, 성공하면 negative result가 contribution이 된다"를 적었습니다. 둘 다 철회합니다 — 전자는 예산 artifact가 섞인 크기이고(§5.6), 후자는 예산만 늘려도 `W`가 6–10배 커지는데 routing 기여가 그대로 0이라 **실험할 이유가 없어졌습니다**(§6.2).

지금 더 정직한 후속 질문: routing이 아니라 **capsule 구조의 무엇이** 회귀를 돕는가. 그리고 공정 예산에서 그 이득(+0.037)이 추가 연산 비용(epoch당 70초 vs 31초, **2.2배**)을 정당화하는가 — 프로브 기준으로는 **그렇지 않습니다.**

정직하게 적어야 할 한계

이 실험들은 **peptide branch 진단**입니다. pseudo-sequence는 실서열이지만, register-shift는 allele을 고정하고 peptide만 교란하므로 **cross-allele 주장의 근거는 아닙니다.**

epoch 예산이 부족합니다 (§5.5). 세 모델 동일하게 부족하므로 비교는 유효.

`scramble_all`은 조성을 보존하지만 **길이/위치 분포까지 완전히 통제하지는 않습니다.** "거친 특징 의존" 가설을 확정하려면 조성 보존 대조군을 하나 더 설계해야 합니다.

core 추적 실험은 **예측 안정성**을 재며, 미지 core에 대한 **일반화를 재지 않습니다** (`core9_in_train_same_allele = 1783`, 설계상 의도).

---

8. 재현 방법

```
cd codes/custom_codes

# round 2 공정 비교 (순차, Phase3는 Phase2 체크포인트로 warm-start하므로 순서 중요)
python train_phase2.py                                          # -> reports_phase2_m2/
python train_phase3.py                                          # -> reports_phase3_m2/
PHASE3_CONFIG=config/phase3_norouting.json python train_phase3.py  # ->
reports_phase3_norouting_m2/

# register-shift (모델별)
python register_shift.py --model phase2 --ckpt-dir reports_phase2_m2/checkpoints
python register_shift.py --model phase3 --ckpt-dir reports_phase3_m2/checkpoints
python register_shift.py --model phase3 --config config/phase3_norouting.json \
                         --ckpt-dir reports_phase3_norouting_m2/checkpoints

# routing 진단
python diag_coupling.py      --ckpt-dir reports_phase3_m2/checkpoints
python diag_routing_iters.py --ckpt-dir reports_phase3_m2/checkpoints
```

경고: Phase2가 실패하면 Phase3를 돌리지 마세요 — 이전 round의 낡은 가중치로 warm-start해서 조용히 오염됩니다. `run_m2.sh`는 `set -e`로 이걸 막습니다.

주요 파일

파일	역할
train_phase2.py	Phase2 학습 + 공유 selection_score() 헬퍼
train_phase3.py	Phase3 학습, PHASE3_CONFIG 환경변수로 config 교체
lomo.py	leave-one-molecule-out, 같은 선택 기준 사용

파일	역할
register_shift.py	register-shift 실험 (--model phase2|phase3)
diag_coupling.py	coupling 텐서 분산 분해
diag_routing_iters.py	학습된 W로 routing loop 재생
config/phase3_norouting.json	routing_iters: 1 ablation

환경

CPU 전용 PyTorch 2.7.1+cpu, 22 logical core (Intel Core Ultra 7 155H). epoch당 26–120s(경합에 따라 변동), 3개 모델 × 5 fold × 50 epoch = 약 **61시간**.

학습을 밤새 돌릴 때 반드시 확인: Windows **Modern Standby(S0)** 가 학습을 멈춥니다. 증상이 교묘합니다 — 프로세스가 죽지 않고 그냥 느려집니다 (11시간 20분에 2 epoch → 176일 완주 예상). Kernel-Power event 506/507로 확인 가능.

```
powercfg /change standby-timeout-ac 0
powercfg /change hibernate-timeout-ac 0
powercfg /change monitor-timeout-ac 0
```

Win+L 잠금과 화면 끄기는 **안전**합니다. 위 설정은 작업이 끝나면 되돌리세요.

---

9. 다음에 할 일

끝난 것 — 수렴 프로브 (2026-11-11, §5.6)

[x] 1 fold × 150 epoch × 3모델. 결론: 50 epoch 예산이 **Phase2를 훨씬 더 심하게** 굶겼고, 그 때문에 §5.2/§5.3의 회귀 격차가 **+0.132 → +0.037로 72% 과대평가**됐습니다.

[x] |w| 질문도 같이 닫혔습니다 — 예산만 늘려도 init의 6–10배까지 자라고, 그 상태에서도 routing 기여는 0. **init 스케일 실험은 불필요해졌습니다**(§6.2).

1순위 — 공정 예산 찾고 전부 재학습 (이게 지금 유일하게 중요한 일)

§5.2/§5.3 숫자는 현재 **논문에 쓸 수 없습니다.** 방향은 살아 있지만 크기가 틀렸습니다.

[ ] **Phase2 평평 지점 확인** — 1 fold × 400 epoch (~3.5 h). 150에서도 단조 상승 중이라 공정 예산이 얼마인지 아직 모릅니다. (진행 중)

[ ] **세 모델 5-fold 전부 재학습**, 그 예산으로. 250 epoch 기준 합계 ~ 54 h (Phase2 11 h + Phase3 24 h + no-routing 19 h). 이게 끝나야 §5의 모든 표와 부트스트랩 CI, §7의 서술을 다시 씁니다.

[ ] 재학습 후 **분류 트레이드오프 재확인.** 프로브에서는 AUROC 격차가 오히려 **벌어졌고**(0.860 vs 0.809) 복합 지표로는 Phase2가 역전했습니다. 사실이면 "회귀는 CapsNet, 분류는 CNN"이 "CapsNet의 이득은 회귀에 한정되고 그마저 작다"로 약해집니다 — 서술 전체가 달라집니다.

3순위 — 재학습 이후로 미룸 (지금 측정해도 예산 바뀌면 다시 해야 합니다)

[ ] **국소 셔플 대조군.** `scramble_all`이 이미 조성을 보존하므로 "조성 보존 대조군"은 새로 얻는 게 없습니다. 필요한 건 3-mer/4-mer 블록 단위 셔플 — 국소 순서는 보존하고 전역 배치만 깨서 Phase3가 쓰는 특징의 **입자 크기**를 재는 설계입니다. 전역에는 둔감한데 국소에 민감하면 "짧은 motif", 둘 다 둔감하면 "정말 조성만".

[ ] per-allele register 측정 — pseudo-sequence가 실서열이므로 이제 가능

- [ ] register-shift 그림에 core-tracking 패널 추가

사람이 해야 하는 일

- [ ] NetMHCIIpan-4.3 라이선스 — DTU 서버가 이 환경에서 접근 불가(HTTP 000)

- [ ] 선행 연구가 core 식별을 어떻게 평가하는지 문헌 조사